

TASK 17:ASSIGNMENT NO 17

Name of Student : Anuja Vilas Wankhade

Domain: Artificial intelligence and machine Learning (AIML)

College Name: Manav School of Polytechnic AKOLA

Branch : Computer Engineering

GitHub Repositories Link:<https://github.com/anujaw193-star/python-project-collection>

Q1.

```
[1] ✓ 0s import numpy as np
import pandas as pd
```

```
[2] ✓ 0s # Load the dataset
df = pd.read_csv("heart - heart.csv")
# Display the first 10 rows
print(df.head(10))
# Display the shape of the dataset
print("Shape of the dataset:", df.shape)
# Display basic information
print("\nDataset Information:")
df.info()
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	\
0	40	M	ATA	140	289	0	Normal	172	
1	49	F	NAP	160	180	0	Normal	156	
2	37	M	ATA	130	283	0	ST	98	
3	48	F	ASY	138	214	0	Normal	108	
4	54	M	NAP	150	195	0	Normal	122	
5	39	M	NAP	120	339	0	Normal	170	
6	45	F	ATA	130	237	0	Normal	170	
7	54	M	ATA	110	208	0	Normal	142	
8	37	M	ASY	140	207	0	Normal	130	
9	48	F	ATA	120	284	0	Normal	120	

	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	N	0.0	Up	0
1	N	1.0	Flat	1
2	N	0.0	Up	0
3	Y	1.5	Flat	1
4	N	0.0	Up	0
5	N	0.0	Up	0
6	N	0.0	Up	0
7	N	0.0	Up	0
8	Y	1.5	Flat	1
9	N	0.0	Up	0

Shape of the dataset: (918, 12)

```

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Age                   918 non-null    int64
1   Sex                   918 non-null    object
2   ChestPainType         918 non-null    object
3   RestingBP             918 non-null    int64
4   Cholesterol            918 non-null    int64
5   FastingBS             918 non-null    int64
6   RestingECG            918 non-null    object
7   MaxHR                 918 non-null    int64
8   ExerciseAngina        918 non-null    object
9   Oldpeak               918 non-null    float64
10  ST_Slope              918 non-null    object
11  HeartDisease          918 non-null    int64
dtypes: float64(1), int64(6), object(5)
memory usage: 86.2+ KB

```

Q2.

```

[4] ✓ Os # Check for missing values in each column
print("\nMissing Values:")
print(df.isnull().sum())

...
Missing Values:
Age                0
Sex                0
ChestPainType      0
RestingBP          0
Cholesterol        0
FastingBS          0
RestingECG         0
MaxHR              0
ExerciseAngina     0
Oldpeak            0
ST_Slope           0
HeartDisease       0
dtype: int64

```

Q3.

```

[7] ✓ Os # Check for duplicate rows
duplicates = df.duplicated().sum()
print("Number of duplicate rows:", duplicates)
# Remove duplicate rows if any exist
if duplicates > 0:
    df = df.drop_duplicates()
    print("Duplicate rows removed.")
else:
    print("No duplicate rows found.")
# Print the new shape of the dataset
print("New shape of the dataset:", df.shape)

...
Number of duplicate rows: 0
No duplicate rows found.
New shape of the dataset: (918, 12)

```

Q4.

```
[9] ✓ Os # Count rows where Cholesterol = 0
Cholesterol_zero = (df['Cholesterol'] == 0).sum()
# Count rows where RestingBP = 0
restingbp_zero = (df['RestingBP'] == 0).sum()
# Print the counts
print("Rows with Cholesterol = 0:", Cholesterol_zero)

print("Rows with RestingBP = 0:", restingbp_zero)
```

... Rows with Cholesterol = 0: 172
Rows with RestingBP = 0: 1

+ Code + Text

Q5.

```
[10] ✓ Os # Statistical summary BEFORE cleaning
print("Before cleaning:")
print(df[['Cholesterol', 'RestingBP']].describe())
# Calculate mean excluding zero values
chol_mean = df.loc[df['Cholesterol'] != 0]['Cholesterol'].mean()
bp_mean = df.loc[df['RestingBP'] != 0, 'RestingBP'].mean()

# Replace 0 values with the calculated means
df['Cholesterol'] = df['Cholesterol'].replace(0, chol_mean)
df['RestingBP'] = df['RestingBP'].replace(0, bp_mean)

# Round both columns to 2 decimal places
df['Cholesterol'] = df['Cholesterol'].round(2)
df['RestingBP'] = df['RestingBP'].round(2)

# Statical summary AFTER cleaning
print("\nAfter cleaning:")
print(df[['Cholesterol', 'RestingBP']].describe())
```

```
... Before cleaning:
      Cholesterol  RestingBP
count    918.000000    918.000000
mean     198.799564    132.396514
std       109.384145     18.514154
min         0.000000         0.000000
25%       173.250000    120.000000
50%       223.000000    130.000000
75%       267.000000    140.000000
max        603.000000    200.000000

After cleaning:
      Cholesterol  RestingBP
count    918.000000    918.000000
mean     244.636253    132.540893
std       53.318029     17.989932
min        85.000000     80.000000
25%       214.000000    120.000000
50%       244.640000    130.000000
75%       267.000000    140.000000
max        603.000000    200.000000
```

Q6.

```
[12] import matplotlib.pyplot as plt

[13] # Data Cleaning
chol_mean = df.loc[df['Cholesterol'] != 0]['Cholesterol'].mean()
bp_mean = df.loc[df['RestingBP'] != 0, 'RestingBP'].mean()

df['Cholesterol'] = df['Cholesterol'].replace(0, chol_mean).round(2)
df['RestingBP'] = df['RestingBP'].replace(0, bp_mean).round(2)

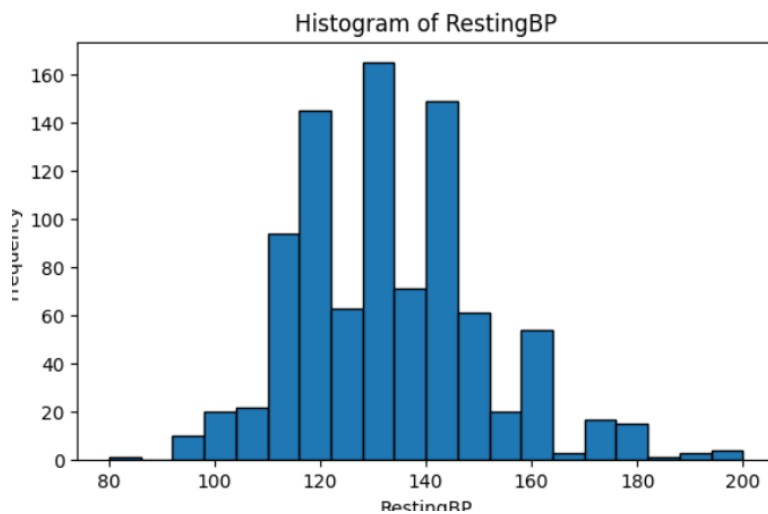
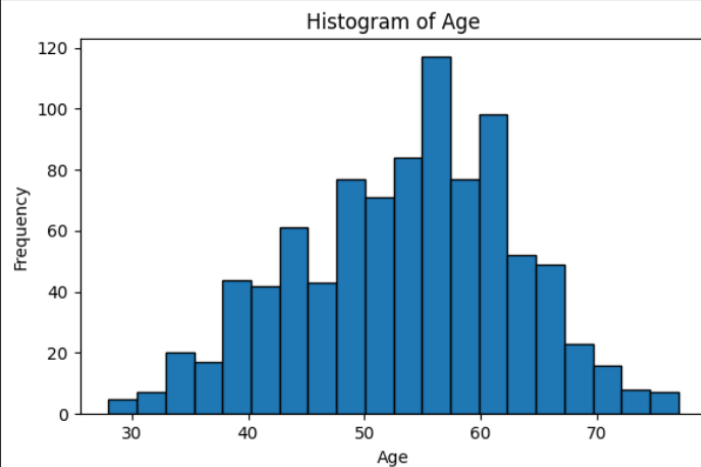
# Function to plot histograms
def plot_histograms(data):
    columns = ['Age', 'RestingBP', 'Cholesterol', 'MaxHR']

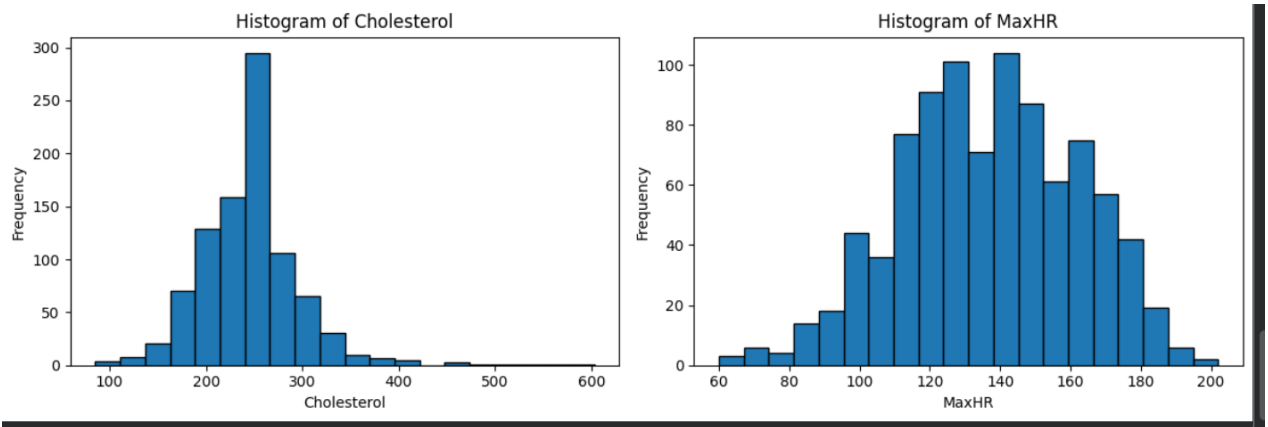
    plt.figure(figsize=(12, 8))

    for i, col in enumerate(columns, 1):
        plt.subplot(2, 2, i)
        plt.hist(data[col], bins=20, edgecolor='black')
        plt.title(f'Histogram of {col}')
        plt.xlabel(col)
        plt.ylabel('Frequency')

    plt.tight_layout()
    plt.show()

# Call the function
plot_histograms(df)
```





Q7.

```
[14] ✓ Os # Identify numerical columns
numerical_columns = df.select_dtypes(include=['int64', 'float64']).columns

# Identify categorical columns
categorical_columns = df.select_dtypes(include=['object']).columns

# Print the columns
print("Numerical Columns:")
print(list(numerical_columns))

print("\nCategorical Columns:")
print(list(categorical_columns))

... Numerical Columns:
['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak', 'HeartDisease']

Categorical Columns:
['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope']
```

Q8.

```
[15] ✓ Os # Perform One-Hot Encoding on categorical columns
df_encoded = pd.get_dummies(df, drop_first=False)

# Print the shape of the encoded dataframe
print("Shape of the encoded dataframe:", df_encoded.shape)

# Display the first 5 rows
print("\nFirst 5 rows of the encoded dataframe:")
print(df_encoded.head())
```

```
... Shape of the encoded dataframe: (918, 21)

First 5 rows of the encoded dataframe:

   Age  RestingBP  Cholesterol  FastingBS  MaxHR  Oldpeak  HeartDisease  \
0   40    140.0     289.0         0      172     0.0         0
1   49    160.0     180.0         0      156     1.0         1
2   37    130.0     283.0         0       98     0.0         0
3   48    138.0     214.0         0     108     1.5         1
4   54    150.0     195.0         0     122     0.0         0

   Sex_F  Sex_M  ChestPainType_ASY  ...  ChestPainType_NAP  ChestPainType_TA  \
0  False   True                False  ...                False            False
1   True  False                False  ...                True             False
2  False   True                False  ...                False            False
3   True  False                True   ...                False            False
4  False   True                False  ...                True             False

   RestingECG_LVH  RestingECG_Normal  RestingECG_ST  ExerciseAngina_N  \
0             False                True           False              True
1             False                True           False              True
2             False                False           True              True
3             False                True           False              False
4             False                True           False              True

   ExerciseAngina_Y  ST_Slope_Down  ST_Slope_Flat  ST_Slope_Up
0                False           False           False         True
1                False           False           True          False
2                False           False           False         True
3                 True           False           True          False
4                False           False           False         True

[5 rows x 21 columns]
```

Q9.

```
[16] ✓ Os
# Clean invalid values
chol_mean = df.loc[df['Cholesterol'] != 0, 'Cholesterol'].mean()
bp_mean = df.loc[df['RestingBP'] != 0, 'RestingBP'].mean()

df['Cholesterol'] = df['Cholesterol'].replace(0, chol_mean).round(2)
df['RestingBP'] = df['RestingBP'].replace(0, bp_mean).round(2)

# One-Hot Encoding
df_encoded = pd.get_dummies(df)

# Print final shape
print("Final Shape of df_encoded:", df_encoded.shape)

# Print all column names
print("\nColumn Names:")
print(df_encoded.columns.tolist())

... Final Shape of df_encoded: (918, 21)

Column Names:
['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak', 'HeartDisease', 'Sex_F', 'Sex_M', 'ChestPainType_ASY', 'ChestPainType_ATA', 'ChestPainType_NAP', 'ChestPainType_TA', 'ExerciseAngina_N', 'ExerciseAngina_Y', 'ST_Slope_Down', 'ST_Slope_Flat', 'ST_Slope_Up']
```

Q10.

Summary

1. ****Invalid Values Found and Fixed:****

- The dataset contained **172 rows where Cholesterol = 0** and **1 row where RestingBP = 0**.
- These values were considered invalid and replaced with the **mean of the respective column**, calculated after excluding zero values.
- Both columns were rounded to **2 decimal places** after cleaning.

2. **Importance of Handling Invalid Values:**

- Invalid values like **Cholesterol = 0** and **RestingBP = 0** are medically unrealistic.
- Such values can distort statistical analysis, reduce model accuracy, and lead to incorrect conclusions.
- Replacing them with appropriate values improves data quality and reliability.

3. **Purpose of One-Hot Encoding:**

- One-Hot Encoding converts categorical variables into numerical (binary) columns.
- This allows machine learning algorithms to process categorical data without assuming any order between categories.

4. **Other Observations:**

- The dataset contained **918 rows** with **no missing values** and **no duplicate records**.
- After cleaning and One-Hot Encoding, the dataset had **918 rows and 21 columns**.
- The dataset is now clean, consistent, and ready for further analysis and machine learning.